

ODESA I. I. MECHNIKOV NATIONAL UNIVERSITY  
FACULTY OF MATHEMATICS, PHYSICS AND INFORMATION TECHNOLOGIES  
DEPARTMENT OF OPTIMAL CONTROL AND ECONOMIC CYBERNETICS

## Computational Geometry and Computer Graphics

### Advanced problem

#### Path planning on Voronoi diagrams

*Uliana Hrab*

6 June 2026

# Contents

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Problem statement</b>                                                     | <b>2</b>  |
| <b>2</b> | <b>Theoretical background</b>                                                | <b>2</b>  |
| 2.1      | The Voronoi diagram . . . . .                                                | 2         |
| 2.2      | The generalised Voronoi diagram for segments . . . . .                       | 2         |
| 2.3      | Relation to path planning . . . . .                                          | 2         |
| <b>3</b> | <b>Solution method</b>                                                       | <b>2</b>  |
| 3.1      | Discretisation and the clearance field . . . . .                             | 2         |
| 3.2      | Maximising the minimum distance (binary search over the threshold) . . . . . | 3         |
| 3.3      | Shortest path inside the safe corridor . . . . .                             | 3         |
| 3.4      | Computational complexity . . . . .                                           | 3         |
| <b>4</b> | <b>Description of the program</b>                                            | <b>3</b>  |
| <b>5</b> | <b>Demonstration and analysis of the results</b>                             | <b>10</b> |
| <b>6</b> | <b>Conclusions</b>                                                           | <b>11</b> |

# 1 Problem statement

The task is to build a trajectory for an agent moving among a set of obstacles so that the **minimum distance** to them is **maximised** (that is, the agent moves as safely as possible), and among all the safest paths the shortest one is selected. As required by the assignment, this is to be done using the generalised Voronoi diagram for line segments together with a shortest-path search on a graph.

## 2 Theoretical background

### 2.1 The Voronoi diagram

For a set of points  $\{p_1, \dots, p_n\}$  the *Voronoi diagram* partitions the plane into regions  $V(p_i)$ , each containing those points of the plane that are closer to  $p_i$  than to any other  $p_j$ . The boundaries between regions consist of points equidistant from the two nearest sites.

### 2.2 The generalised Voronoi diagram for segments

If the obstacles are not points but *segments* (the sides of polygons), we obtain the *generalised Voronoi diagram*: its edges consist of points equidistant from the two nearest obstacles. These edges form the “spine” that runs exactly halfway between the obstacles and is therefore the locus of points of maximum safety — the so-called *medial axis* of the free space.

### 2.3 Relation to path planning

Introduce the *clearance field* (the distance to the nearest obstacle)

$$c(\mathbf{x}) = \min_k \text{dist}(\mathbf{x}, s_k),$$

where the  $s_k$  are the obstacle segments. The ridges of this field (local maxima along the direction between obstacles) are exactly the generalised Voronoi diagram. If the path is routed through regions with large  $c(\mathbf{x})$ , the agent keeps away from the obstacles. Formally we look for a path  $\gamma$  from the start to the goal that attains

$$\max_{\gamma} \min_{\mathbf{x} \in \gamma} c(\mathbf{x}),$$

that is, one that maximises the minimum distance to the obstacles along the whole route (the *widest path* problem, also known as the bottleneck path problem).

## 3 Solution method

### 3.1 Discretisation and the clearance field

The space is partitioned into a uniform grid of  $GW \times GH$  cells. For the centre of each cell the clearance is computed — the minimum distance to all obstacle segments. The distance from a point  $\mathbf{p}$  to a segment  $\overline{\mathbf{a}\mathbf{b}}$  is computed by projection:

$$t = \text{clamp}\left(\frac{(\mathbf{p} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|^2}, 0, 1\right), \quad \text{dist} = \|\mathbf{p} - (\mathbf{a} + t(\mathbf{b} - \mathbf{a}))\|.$$

The clearance is visualised with a colour map (a heat map): red near the obstacles, and through yellow and green to blue far away from them.

### 3.2 Maximising the minimum distance (binary search over the threshold)

The widest-path problem is solved as follows. Fix a threshold  $T$  and keep only the cells with  $c \geq T$ . The start and the goal are connected for this  $T$  if and only if there exists a path whose minimum clearance is at least  $T$ . Connectivity is checked with a breadth-first search (BFS). The connectivity predicate is monotone in  $T$ , so the **largest** value  $T^*$  (the safest corridor) is found by *binary search* over  $T$ . The resulting  $T^*$  is precisely the sought  $\max_{\gamma} \min_{\mathbf{x}} c$ .

### 3.3 Shortest path inside the safe corridor

Among all the paths whose minimum clearance is  $\geq T^*$ , the shortest one is chosen. To do this, Dijkstra's algorithm is run on the sub-grid of cells with  $c \geq T^*$  using 8-connectivity and Euclidean edge weights (1 for orthogonal neighbours,  $\sqrt{2}$  for diagonal ones). The path is reconstructed from the array of predecessors.

### 3.4 Computational complexity

Let the grid contain  $N = GW \cdot GH$  cells and let there be  $m$  obstacle segments. Building the clearance field costs  $O(Nm)$ . A single BFS costs  $O(N)$ , and the binary search adds a factor of  $O(\log(1/\varepsilon))$ . Dijkstra's algorithm with a binary heap costs  $O(N \log N)$ . The overall complexity is  $O(Nm + N \log N)$ .

## 4 Description of the program

The program is written in C++ using FreeGLUT. Obstacles are stored as a vector of segments (`std::vector<Seg>`); the outer walls are added separately so that the path is not pressed against the edges of the window. The key functions are:

- `distPointSeg` — distance from a point to a segment;
- `computeClearance` — filling in the clearance field for the whole grid;
- `connected` — checking whether the start and the goal are connected for a given threshold (BFS);
- `planPath` — binary search for the threshold  $T^*$  and the call to the path search;
- `shortestPath` — Dijkstra's algorithm inside the safe corridor.

Controls: left click — set the start, right click — set the goal, **R** — generate new obstacles, **Esc** — quit. After any change the route is recomputed, and the minimum clearance along the path found is printed in the upper corner.

```

// Planning is done in two steps:
// 1. binary search for the largest clearance threshold T such that start and
//    goal are still connected through cells with clearance >= T -> this
//    maximizes the minimum distance to the obstacles (the bottleneck);
// 2. Dijkstra (8-connected) inside that safe corridor -> shortest path among
//    the safest ones.
//
// Controls: left click - set start, right click - set goal,
//           R - regenerate obstacles, Esc - quit.
#include <GL/freeglut.h>
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <queue>
#include <vector>

const int WIN_W = 960;
const int WIN_H = 720;
const int CELL = 6;
const int GW = WIN_W / CELL; // grid width (cells)
const int GH = WIN_H / CELL; // grid height (cells)

struct Vec { float x; float y; };
struct Seg { Vec a; Vec b; };

std::vector<Seg> obstacles;
std::vector<float> clearance(GW * GH, 0.0f);
float maxClearance = 1.0f;

Vec startP = { 90.0f, 90.0f };
Vec goalP = { (float)WIN_W - 90.0f, (float)WIN_H - 90.0f };
std::vector<Vec> path;
float pathBottleneck = 0.0f;

inline int idx(int i, int j) { return j * GW + i; }
inline Vec cellCenter(int i, int j) { return { (i + 0.5f) * CELL, (j + 0.5f) * CELL }; }

float distPointSeg(Vec p, const Seg& s)
{
    float vx = s.b.x - s.a.x, vy = s.b.y - s.a.y;
    float wx = p.x - s.a.x, wy = p.y - s.a.y;
    float len2 = vx * vx + vy * vy;
    float t = (len2 > 0.0f) ? (wx * vx + wy * vy) / len2 : 0.0f;
    if (t < 0.0f) t = 0.0f;
    if (t > 1.0f) t = 1.0f;
    float cx = s.a.x + t * vx, cy = s.a.y + t * vy;
    float dx = p.x - cx, dy = p.y - cy;
    return std::sqrt(dx * dx + dy * dy);
}

void addRect(float x, float y, float w, float h)
{
    Vec p1 = { x, y }, p2 = { x + w, y }, p3 = { x + w, y + h }, p4 = { x, y + h };

```

```

    obstacles.push_back({ p1, p2 });
    obstacles.push_back({ p2, p3 });
    obstacles.push_back({ p3, p4 });
    obstacles.push_back({ p4, p1 });
}

void buildObstacles()
{
    obstacles.clear();
    // outer walls so the path keeps away from the borders
    addRect(0.0f, 0.0f, (float)WIN_W, (float)WIN_H);
    // random rectangular obstacles
    int n = 5 + std::rand() % 3;
    for (int k = 0; k < n; ++k)
    {
        float w = 70.0f + std::rand() % 120;
        float h = 70.0f + std::rand() % 120;
        float x = 120.0f + std::rand() % (int)(WIN_W - 240 - w);
        float y = 100.0f + std::rand() % (int)(WIN_H - 200 - h);
        addRect(x, y, w, h);
    }
}

void computeClearance()
{
    maxClearance = 1.0f;
    for (int j = 0; j < GH; ++j)
        for (int i = 0; i < GW; ++i)
        {
            Vec c = cellCenter(i, j);
            float best = 1e9f;
            for (const Seg& s : obstacles)
                best = std::min(best, distPointSeg(c, s));
            clearance[idx(i, j)] = best;
            maxClearance = std::max(maxClearance, best);
        }
}

inline void worldToCell(Vec p, int& i, int& j)
{
    i = (int)(p.x / CELL);
    j = (int)(p.y / CELL);
    if (i < 0) i = 0; if (i >= GW) i = GW - 1;
    if (j < 0) j = 0; if (j >= GH) j = GH - 1;
}

// BFS: are start and goal connected through cells with clearance >= thr?
bool connected(float thr, int si, int sj, int gi, int gj)
{
    if (clearance[idx(si, sj)] < thr || clearance[idx(gi, gj)] < thr)
        return false;
    std::vector<char> seen(GW * GH, 0);
    std::queue<int> q;
    seen[idx(si, sj)] = 1;
    q.push(idx(si, sj));
    while (!q.empty())

```

```

{
    int cur = q.front(); q.pop();
    int ci = cur % GW, cj = cur / GW;
    if (ci == gi && cj == gj) return true;
    for (int dj = -1; dj <= 1; ++dj)
        for (int di = -1; di <= 1; ++di)
        {
            if (di == 0 && dj == 0) continue;
            int ni = ci + di, nj = cj + dj;
            if (ni < 0 || nj < 0 || ni >= GW || nj >= GH) continue;
            int nk = idx(ni, nj);
            if (seen[nk] || clearance[nk] < thr) continue;
            seen[nk] = 1;
            q.push(nk);
        }
    }
    return false;
}

// Dijkstra (8-connected, Euclidean weights) on cells with clearance >= thr.
void shortestPath(float thr, int si, int sj, int gi, int gj)
{
    path.clear();
    const float INF = 1e18f;
    std::vector<float> dist(GW * GH, INF);
    std::vector<int> prev(GW * GH, -1);
    typedef std::pair<float, int> PQ;
    std::priority_queue<PQ, std::vector<PQ>, std::greater<PQ>> pq;

    int s = idx(si, sj), g = idx(gi, gj);
    dist[s] = 0.0f;
    pq.push({ 0.0f, s });

    while (!pq.empty())
    {
        PQ top = pq.top(); pq.pop();
        float d = top.first; int cur = top.second;
        if (d > dist[cur]) continue;
        if (cur == g) break;
        int ci = cur % GW, cj = cur / GW;
        for (int dj = -1; dj <= 1; ++dj)
            for (int di = -1; di <= 1; ++di)
            {
                if (di == 0 && dj == 0) continue;
                int ni = ci + di, nj = cj + dj;
                if (ni < 0 || nj < 0 || ni >= GW || nj >= GH) continue;
                int nk = idx(ni, nj);
                if (clearance[nk] < thr) continue;
                float step = (di != 0 && dj != 0) ? 1.41421356f : 1.0f;
                float nd = d + step;
                if (nd < dist[nk])
                {
                    dist[nk] = nd;
                    prev[nk] = cur;
                    pq.push({ nd, nk });
                }
            }
    }
}

```

```

    }
}

if (dist[g] >= INF) return;
for (int cur = g; cur != -1; cur = prev[cur])
{
    int ci = cur % GW, cj = cur / GW;
    path.push_back(cellCenter(ci, cj));
}
std::reverse(path.begin(), path.end());
}

void planPath()
{
    int si, sj, gi, gj;
    worldToCell(startP, si, sj);
    worldToCell(goalP, gi, gj);

    // maximize the minimum clearance (bottleneck) via binary search
    float lo = 0.0f;
    float hi = std::min(clearance[idx(si, sj)], clearance[idx(gi, gj)]);
    if (!connected(0.5f, si, sj, gi, gj)) { path.clear(); pathBottleneck = 0.0f; return; }
    for (int it = 0; it < 40; ++it)
    {
        float mid = 0.5f * (lo + hi);
        if (connected(mid, si, sj, gi, gj)) lo = mid; else hi = mid;
    }
    pathBottleneck = lo;
    // shortest path inside the safe corridor (use slightly relaxed threshold)
    shortestPath(std::max(0.0f, lo - 0.01f), si, sj, gi, gj);
}

void recompute()
{
    computeClearance();
    planPath();
    glutPostRedisplay();
}

void heatColor(float t)    // t in [0,1]: 0 = near obstacle (red), 1 = far (blue)
{
    float r, g, b;
    if (t < 0.5f) { r = 1.0f; g = t * 2.0f; b = 0.0f; }           // red -> yellow
    else         { float u = (t - 0.5f) * 2.0f; r = 1.0f - u; g = 1.0f; b = u; } // yellow ->
    green -> blue
    glColor3f(r * 0.85f + 0.1f, g * 0.7f + 0.15f, b * 0.8f + 0.15f);
}

void drawHeatmap()
{
    glBegin(GL_QUADS);
    for (int j = 0; j < GH; ++j)
        for (int i = 0; i < GW; ++i)
        {
            float t = clearance[idx(i, j)] / maxClearance;
            heatColor(t);
        }
}

```

```

        float x = i * (float)CELL, y = j * (float)CELL;
        glVertex2f(x, y);
        glVertex2f(x + CELL, y);
        glVertex2f(x + CELL, y + CELL);
        glVertex2f(x, y + CELL);
    }
    glEnd();
}

void drawObstacles()
{
    glColor3f(0.1f, 0.1f, 0.12f);
    glLineWidth(2.5f);
    glBegin(GL_LINES);
    for (const Seg& s : obstacles)
    {
        glVertex2f(s.a.x, s.a.y);
        glVertex2f(s.b.x, s.b.y);
    }
    glEnd();
}

void drawDisc(Vec c, float r, float cr, float cg, float cb)
{
    glColor3f(cr, cg, cb);
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(c.x, c.y);
    for (int a = 0; a <= 24; ++a)
    {
        float t = a * 2.0f * 3.1415926f / 24.0f;
        glVertex2f(c.x + r * std::cos(t), c.y + r * std::sin(t));
    }
    glEnd();
}

void drawText(float x, float y, const char* s)
{
    glRasterPos2f(x, y);
    for (const char* p = s; *p; ++p)
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *p);
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    drawHeatmap();
    drawObstacles();

    if (path.size() >= 2)
    {
        glColor3f(0.05f, 0.05f, 0.05f);
        glLineWidth(6.0f);
        glBegin(GL_LINE_STRIP);
        for (Vec p : path) glVertex2f(p.x, p.y);
        glEnd();
        glColor3f(1.0f, 1.0f, 1.0f);
    }
}

```

```

        glLineWidth(3.0f);
        glBegin(GL_LINE_STRIP);
        for (Vec p : path) glVertex2f(p.x, p.y);
        glEnd();
    }

    drawDisc(startP, 9.0f, 0.0f, 0.8f, 0.2f);    // start - green
    drawDisc(goalP, 9.0f, 0.9f, 0.1f, 0.1f);    // goal - red

    glColor3f(0.0f, 0.0f, 0.0f);
    char buf[128];
    std::snprintf(buf, sizeof(buf), "Min clearance on path: %.1f px", pathBottleneck);
    drawText(12.0f, WIN_H - 22.0f, buf);
    if (path.empty())
        drawText(12.0f, WIN_H - 46.0f, "No path found");
    drawText(12.0f, 14.0f, "L-click: start  R-click: goal  R: new obstacles  Esc: quit");

    glutSwapBuffers();
}

void mouse(int button, int state, int x, int y)
{
    if (state != GLUT_DOWN) return;
    Vec p = { (float)x, (float)(WIN_H - y) };    // flip y to OpenGL coords
    if (button == GLUT_LEFT_BUTTON) startP = p;
    if (button == GLUT_RIGHT_BUTTON) goalP = p;
    planPath();
    glutPostRedisplay();
}

void keyboard(unsigned char key, int x, int y)
{
    if (key == 27) std::exit(0);
    if (key == 'r' || key == 'R')
    {
        buildObstacles();
        recompute();
    }
}

void initScene()
{
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, WIN_W, 0.0, WIN_H);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    std::srand((unsigned)std::time(nullptr));
    buildObstacles();
    computeClearance();
    planPath();
}

int main(int argc, char** argv)
{

```

```

glutInit(&argc, argv);
glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
glutInitWindowSize(WIN_W, WIN_H);
glutInitWindowPosition(80, 80);
glutCreateWindow("Bonus - Voronoi-based Path Planning");

initScene();
glutDisplayFunc(display);
glutMouseFunc(mouse);
glutKeyboardFunc(keyboard);
glutMainLoop();
return 0;
}

```

## 5 Demonstration and analysis of the results

The heat map shows the clearance field: the warmer the colour, the closer the obstacle. The white route runs mostly along the “blue” (safest) areas — the ridges of the generalised Voronoi diagram — staying halfway between the obstacles. The green marker is the start, the red one is the goal. When the points are moved or new obstacles are generated with the R key, the path is recomputed instantly, which demonstrates the algorithm working on a range of configurations.

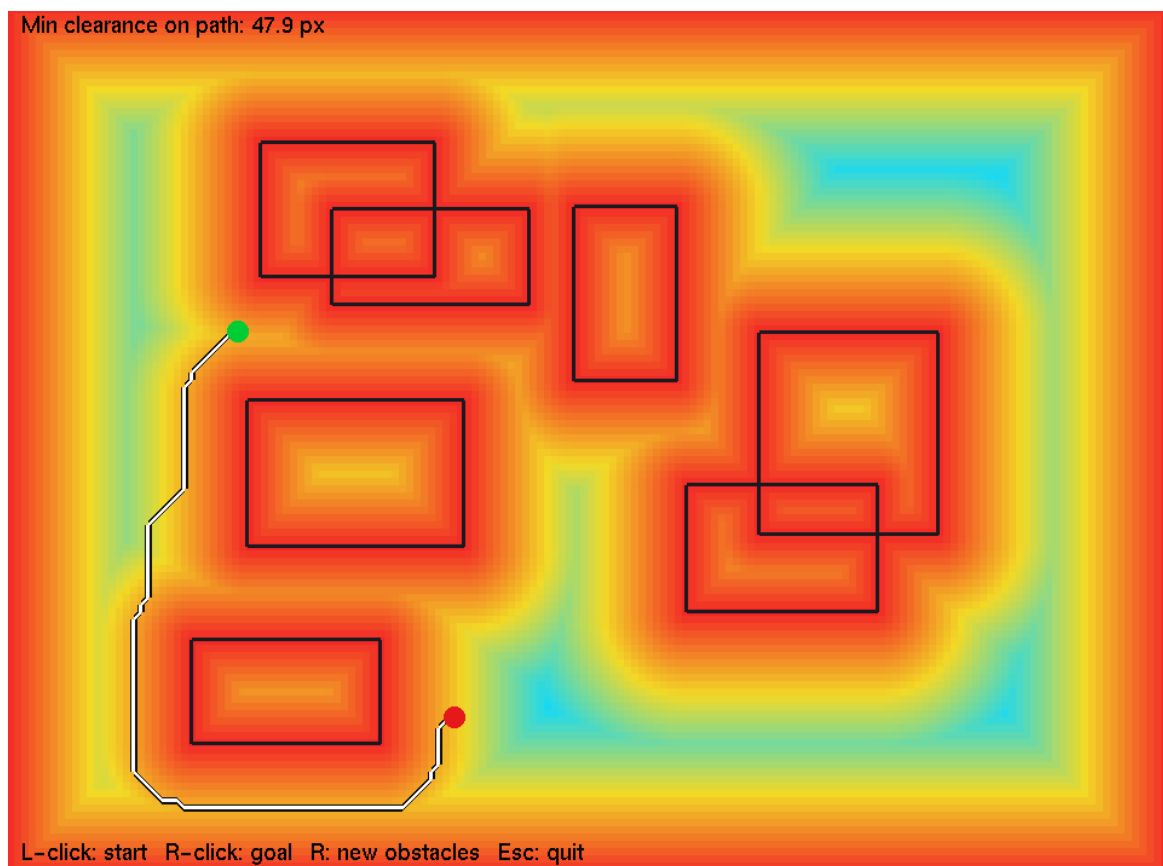


Figure 1: Path planning: the clearance field, the obstacles and the safe route

## 6 Conclusions

A path planner based on the generalised Voronoi diagram has been implemented. The clearance field, built as the distance to the nearest obstacle segments, defines the medial axis of the free space; binary search over the clearance threshold maximises the minimum distance to the obstacles (the widest path), and Dijkstra's algorithm selects the shortest among the safest paths. The approach is robust, works with arbitrary sets of segments and clearly demonstrates the connection between Voronoi diagrams and the problem of safe agent motion.